



Evolución de los Diagramas T

Proyecto de Carrera: Ingeniería en Informática · Asignatura: Lenguajes y Compiladores
Profesor: Márquez Félix · Ciudad Guayana, mayo de 2026

Definición



En el ámbito educativo

Organizador gráfico de dos columnas que permite comparar dos perspectivas sobre un mismo tema. Una barra horizontal superior con el título y una barra vertical central separa los campos de escritura (Miro, 2024; UNAM).

En informática teórica

Los **T-diagrams** o "tombstone diagrams" representan procesadores de lenguajes (compiladores, intérpretes, ensambladores) anotados con tres lenguajes: **fuentes** (izquierda), **objetivo** (derecha) e **implementación** (base). Permiten razonar sobre cadenas de transformaciones entre lenguajes (Wickerson, 2020).



EVOLUCIÓN HISTÓRICA

De UNCOL a la Consolidación Académica

1

1958 — Proyecto UNCOL

Strong, Sammet y Mock propusieron un lenguaje intermedio universal: $n+m$ traductores en lugar de $n \times m$.

2

1961 — Harvey Bratman

Formalizó la "T" con tres componentes: Source (izquierda), Target (derecha), Implementation (base).

3

1970 — McKeeman et al.

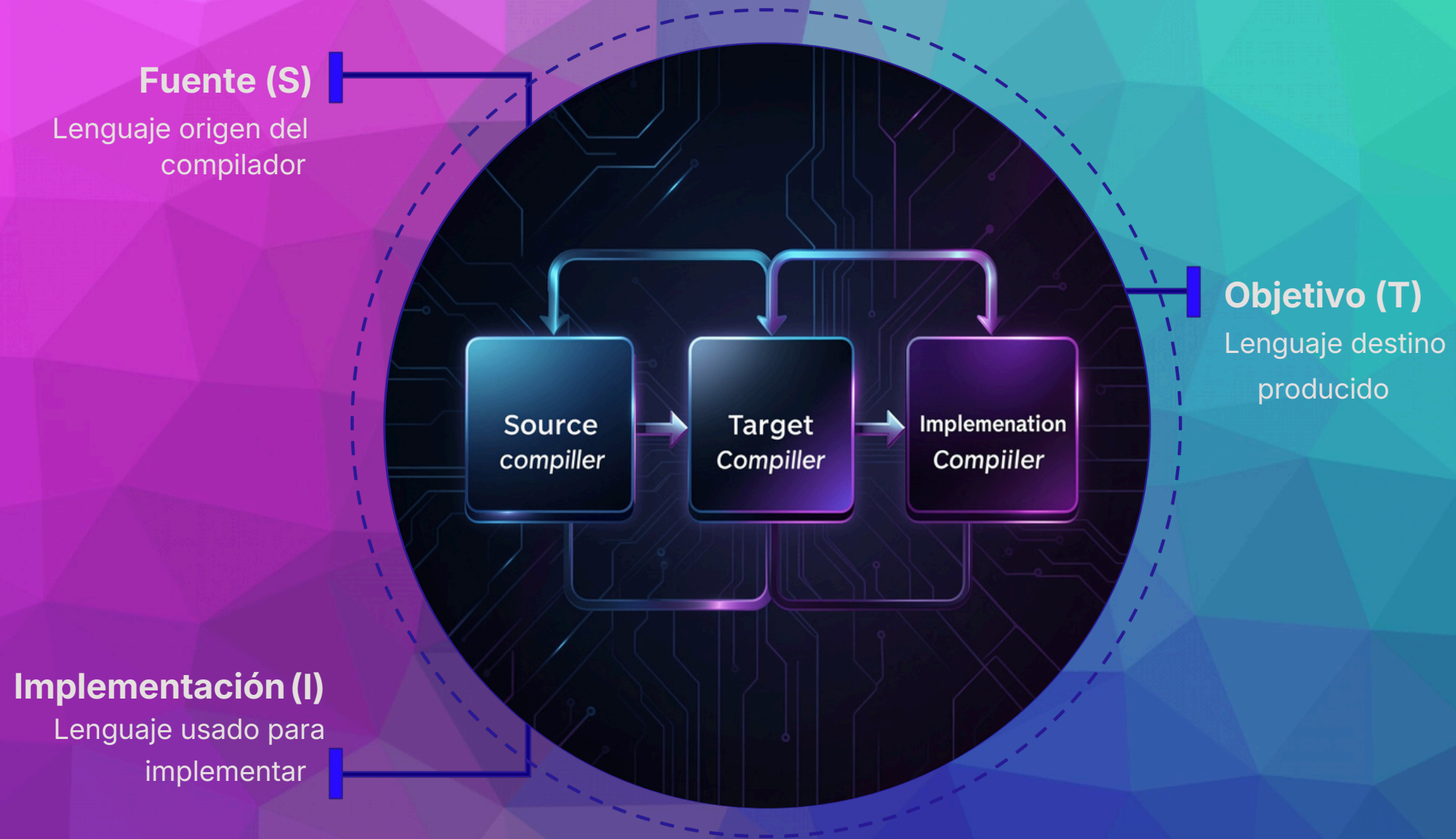
En *A Compiler Generator*, demostraron que las T podían "encajarse" para formalizar el bootstrapping.

4

1986–Presente

Aho, Sethi y Ullman (*Libro del Dragón*) popularizaron los diagramas T para explicar front-end y back-end de compiladores modernos (LLVM, JVM).

Estructura Básica del Diagrama T



En compiladores, la barra horizontal superior contiene el lenguaje fuente (izquierda) y el lenguaje objetivo (derecha); la base indica el lenguaje de implementación. Estas piezas se "encajan" formando cadenas de compilación o traducción entre lenguajes (Wickerson, 2020).

Tipos de Diagrama T

Compilador Convencional

Traduce alto nivel a código máquina nativo. $I = T$ = arquitectura de ejecución.

Cross-Compiler

Ejecuta en máquina anfitriona, genera código para arquitectura distinta ($I \neq T$).

Intérprete

Procesa código fuente en tiempo real; elimina el brazo derecho (sin archivo ejecutable).

Transpilador

Traduce alto nivel a alto nivel (ej. TypeScript $\rightarrow$ JavaScript).

Autohospedaje

El compilador está escrito en su propio lenguaje ($S = I$). Base del bootstrapping.

Máquinas Virtuales

Modela cómo el bytecode se ejecuta sobre una VM (ej. Java/JVM).

Ventajas del Diagrama T



Abstracción Visual

Convierte procesos técnicos complejos en representaciones geométricas sencillas: qué entra, qué sale y qué se necesita.



Bootstrapping

Desglosa el proceso de autocompilación en etapas visuales, resolviendo la paradoja del huevo y la gallina (Wirth, 1996).



Validación Lógica

La "regla de encaje" detecta errores de diseño antes de programar: la base y los brazos deben coincidir.



Portabilidad

Ilustra que solo se modifica el brazo derecho para retargeting, reutilizando el analizador léxico y sintáctico.

Desventajas y Limitaciones

Efecto "Caja Negra"

Trata al compilador como unidad indivisible; no ofrece visibilidad sobre fases internas (análisis, optimización).

Inadecuado para JIT y Reflexión

Diseñado para compilación AOT; no captura traducción dinámica en tiempo de ejecución (Java, C#, JavaScript).

Sin Métricas de Rendimiento

Indica viabilidad lógica, pero no eficiencia del código, RAM ni tiempo de CPU (Aho et al., 2008).

Ambigüedad en Nube y Multi-lenguaje

Dificulta modelar infraestructura virtualizada (contenedores, serverless) y proyectos con múltiples lenguajes.



Metodología de Modelado



Se fundamenta en teoría de tipos, semántica denotacional y polimorfismo (Pierce, 2002). Herramientas: **Haskell/GHC**, **Agda & Coq**, **Alloy Analyzer**. Buenas prácticas: tipos mínimos, nombrado semántico, modularización progresiva y revisión por pares (Jackson, 2012).

Comparación con Otros Diagramas

Criterio	Diagrama T	ER	UML	Flujo/DFD
Propósito	Tipos y transformaciones	Entidades y relaciones	Diseño OO	Procesos y decisiones
Verificación	Formal (teoría de tipos)	Parcial	Parcial (OCL)	No formal
Compiladores	Fundamental	Muy bajo	Bajo-medio	Bajo
Aprendizaje	Media-alta	Media	Alta	Bajo

¿Cuándo usar Diagramas T? Sistemas con verificación formal de tipos, lenguajes estáticos (Haskell, Rust, TypeScript) y dominios ricos en transformaciones: compiladores, parsers, pipelines.

Aplicaciones y Tendencias Futuras



Aplicaciones Clave

- **Compiladores:** GCC, Clang (front-end/back-end)
- **Cross-compiling:** arm-none-eabi-gcc para embebidos
- **Bootstrapping:** Pascal autohospedado (Wirth)
- **Máquinas virtuales:** Java/JVM, .NET CLR
- **Transpiladores:** TypeScript, Babel, Kotlin
- **Emuladores:** Game Boy (Z80 en C++)
- **Nube:** WASM en contenedores Docker

Tendencias Futuras

WebAssembly retomala idea UNCOL como lenguaje intermedio universal. La IA se integra como optimizador intermedio, y herramientas interactivas permiten manipular cadenas de compilación visualmente (Aho et al., 2008; Mozilla, 2023).